

1. Host Simulator 구성 정리

[목차]

- Host Simulator
- Local Agent
- Relay R1
- Relay R2
- Monitor
- Controller / UI

1. Host Simulator

1-1. Host Simulator의 정체

Host Simulator는 **가짜 서버(host)** 를 흉내 내는 프로세스

이 프로젝트에서 실제 OS 상태를 읽는 대신,

이 프로세스가 **CPU / Memory / Disk / Service / Network Latency** 같은 상태를 만들어낸다.

[이 프로세스의 역할]

- 장애를 직접 해결하는 것 아님
- 메시지를 네트워크로 전달하는 것 아님
- **이벤트의 원천 상태를 생성하는 것**

1-2. 왜 필요한가

실제 서버를 망가뜨리면서 실험하기는 어렵다.

그래서 Host Simulator가 다음을 대신한다.

- 정상 상태 유지
- 특정 장애 발생
- 장애 지속
- 복구 상태 전환

즉 네 프로젝트에서 Host Simulator는

“호스트 측 상태 생성기” 다.

1-3. 이 프로세스가 담당하는 요소

Host Simulator 안에는 최소한 아래 요소가 있어야 한다.

상태 요소

- CPU usage

- Memory usage
- Disk usage
- Service state
- Network latency state
- Host alive/dead state

시간 요소

- 상태 갱신 주기
- 장애 지속 시간
- 복구 시점

제어 요소

- 수동 장애 주입
- 자동 랜덤 변화
- reset

1-4. Host Simulator의 목적

목적은 3개다.

1. 정상 상태를 계속 생성

예:

- CPU 18%
- Memory 42%
- Service UP
- Latency NORMAL

2. 특정 장애 상태를 강제로 발생

예:

- CPU Spike
- Service Down
- Latency High

3. 장애가 복구된 상태를 다시 생성

예:

- Service DOWN → UP
- CPU 95% → 23%

즉 Agent가 감지할 수 있도록

상태 변화가 있어야 한다.

1-5. 입력 / 출력

Host Simulator가 받는 입력은 보통 두 종류다.

A. [입력] 내부 주기 타이머

- 매 1초 또는 2초마다 상태 갱신

B. [입력] Controller의 제어 명령

예:

- `inject_cpu_spike`
- `inject_service_down`
- `inject_latency_high`
- `recover_service`
- `reset_host`

C. 출력

출력은 “메시지 전송”이 아니라 **현재 host 상태 데이터**다.

이 상태는 Local Agent가 읽는다.

```
{
  "host_id": "host-1",
  "timestamp": "2026-03-19T10:00:01",
  "cpu": 24,
  "memory": 48,
  "disk": 61,
  "service": "UP",
  "latency": "NORMAL",
  "alive": true
}
```

즉 Host Simulator는 **상태를 제공하는 프로세스**다.

1-6. 내부적으로 가져야 하는 상태값

실제 구현 기준으로는 아래 필드가 있으면 충분하다.

```
host_id
cpu_usage
memory_usage
disk_usage
service_state
latency_state
alive_state
last_update_time
```

```
fault_mode
fault_end_time
```

각 의미는 이렇다.

`host_id`

- 이 host를 구분하는 식별자
- 예: `host-1`

`cpu_usage`

- 0~100 정수 또는 float

`memory_usage`

- 0~100

`disk_usage`

- 0~100

`service_state`

`UP` / `DOWN`

`latency_state`

`NORMAL` / `HIGH`

`alive_state`

`true` / `false`

`fault_mode`

- 현재 어떤 장애가 걸려 있는지

예:

- `NONE`
- `CPU_SPIKE`
- `SERVICE_DOWN`
- `LATENCY_HIGH`

`fault_end_time`

- 장애가 자동 종료될 시점
- 없으면 수동 복구 전까지 유지

1-7. Host Simulator가 제공해야 하는 기능

기능 1. 상태 초기화

목적 : 프로세스 시작 시 정상 상태를 만든다.

작동 방식

- 시작할 때 아래 같은 기본값을 넣는다.

```
cpu_usage = 15
memory_usage = 40
disk_usage = 55
```

```
service_state = UP
latency_state = NORMAL
alive_state = true
fault_mode = NONE
```

실제 작동

- 프로그램 실행
- 내부 상태 초기화
- 이후 주기적으로 값 갱신 시작

기능 2. 주기적 상태 갱신

목적 : host가 살아 있는 것처럼 상태를 조금씩 변하게 한다.

작동 방식

- 매 tick마다 값이 조금 변한다.

예:

- CPU: 15 → 19 → 17 → 21
- Memory: 40 → 41 → 42
- Disk: 거의 천천히 증가
- Service: 기본적으로 UP 유지

이때 장애가 없으면 **작은 랜덤 변화**만 준다.

실제 작동

- 예를 들어 1초마다:

```
cpu += random(-3 ~ +3)
memory += random(-2 ~ +2)
disk += random(0 ~ +1)
```

단 범위는 0~100으로 clamp 한다.

기능 3. 장애 주입

목적 : 실험용으로 특정 비정상 상태를 강제로 만든다.

지원해야 할 장애 : 최소한 아래는 있어야 한다.

- CPU Spike
- Service Down
- Latency High
- Host Recovery

필요하면 추가:

- Memory Pressure
- Host Unreachable

작동방식

- Controller에서 명령이 오면 내부 상태를 즉시 바꾼다.

예:

- `inject_cpu_spike(duration=10)`
- `inject_service_down()`
- `inject_latency_high(duration=15)`

기능 4. 장애 지속 관리

목적 : 장애가 영구인지, 잠깐인지 제어한다.

작동방식 : 예를 들어 CPU Spike 10초짜리면:

- 현재 시각 + 10초를 `fault_end_time` 에 저장
- 주기 갱신 때 현재 시각과 비교
- 시간이 지나면 자동 recovery

실제 작동

```
if now >= fault_end_time:
    clear fault
    restore normal state
```

기능 5. 복구 처리

목적 : 장애가 끝난 뒤 정상 상태로 되돌린다.

작동방식

예:

- CPU 95% → 20~30 범위
- Service DOWN → UP
- Latency HIGH → NORMAL

실제작동

- 복구 시점에 상태를 정상 범위로 되돌리고, `fault_mode = NONE` 으로 만든다.

기능 6. 상태 조회 제공

목적

Local Agent가 현재 상태를 읽을 수 있게 한다.

작동 방식

방법은 두 가지 중 하나다.

방식 A. Agent가 polling

- Agent가 주기적으로 Host Simulator에 요청
- Host Simulator는 현재 상태 반환

방식 B. Host Simulator가 상태 공유

- shared object / local socket / HTTP endpoint 등으로 상태 노출

현재 프로젝트에는 **polling 방식**이 가장 단순하다.

기능 7. reset

목적

실험 초기화

작동 방식

- fault 제거
- 상태 정상값으로 복원
- 시뮬레이션 시간 초기화
- 통계값 초기화

1-8. 장애별 내부 동작

이 부분이 중요하다.

Host Simulator가 장애마다 실제로 뭘 하는지 정리한다.

A. CPU Spike

목적

CPU 과부하 상황 시뮬레이션

내부 동작

- `fault_mode = CPU_SPIKE`
- `cpu_usage = 90~100`
- 나머지 값은 큰 변화 없이 유지 가능

실제 작동 예

정상:
cpu=21

CPU Spike 주입 후:
cpu=95
cpu=97
cpu=94

```
...
복구 후:
cpu=24
```

Agent 입장에서 보이는 것

- CPU가 임계치 초과
- `CPU_SPIKE` 이벤트 생성 대상

B. Service Down

목적

서비스 장애 상황 시뮬레이션

내부 동작

- `fault_mode = SERVICE_DOWN`
- `service_state = DOWN`

선택적으로:

- CPU는 낮아질 수도 있음
- latency는 그대로일 수도 있음

실제 작동 예

```
정상:
service=UP

장애 후:
service=DOWN

복구 후:
service=UP
```

Agent 입장에서 보이는 것

- 서비스 비가동
- `SERVICE_DOWN` 이벤트 생성 대상

C. Latency High

목적

호스트 쪽 네트워크 상태 악화 시뮬레이션

내부 동작

- `fault_mode = LATENCY_HIGH`

- `latency_state = HIGH`

선택적으로 숫자형 latency도 둘 수 있다.

예:

- `latency_ms = 20 → 300`

Agent 입장에서 보이는 것

- `NETWORK_LATENCY_HIGH` 이벤트 가능

D. Recovery

목적

장애 종료 이후 정상 복귀를 명확히 보여주기 위함

내부 동작

- fault 해제
- 정상 범위 상태로 회복

Agent 입장에서 보이는 것

- 복구 이벤트 생성 가능
- 예: `SERVICE_RECOVERED`

1-9. 실제 작동 순서

이제 Host Simulator만 떼어 놓고, 실제로 어떻게 움직이는지 흐름으로 보자.

정상 동작 흐름

1. Host Simulator 시작
2. 기본 정상 상태 초기화
3. 매 1초마다 상태 갱신
4. CPU/Memory/Disk 값이 조금씩 변화
5. Local Agent가 현재 상태를 읽음
6. 큰 변화 없으면 이벤트 없음

장애 주입 흐름

1. Controller가 CPU Spike 명령 전송
2. Host Simulator가 `fault_mode=CPU_SPIKE` 설정
3. `cpu_usage`를 95 전후로 고정
4. Agent가 다음 polling에서 이를 감지
5. Agent가 `CPU_SPIKE` 이벤트 생성
6. 장애 종료 또는 `recovery` 명령까지 상태 유지

자동 복구 흐름

1. CPU Spike 10초짜리 주입
2. Host Simulator가 fault_end_time 저장
3. 10초 동안 cpu_usage 고정
4. 시간 만료
5. fault_mode 해제
6. cpu_usage 정상 범위로 복귀
7. Agent가 recovery 상태 감지

1-10. 실제 구현 시 권장 인터페이스

Host Simulator를 다른 프로세스와 연결하려면 인터페이스가 필요하다.

초기 구현에서는 아래 정도가 현실적이다.

상태 조회 API

- `get_status()`
- 현재 상태 반환

제어 API

- `inject_fault(type, duration)`
- `recover()`
- `reset()`

내부 루프

- `tick()`
- 1초마다 상태 갱신

1-11. 아주 단순한 내부 의사코드

코드는 아직 아니고, 구조만 보자.

```
start():
    initialize_state()
    while running:
        tick()
        sleep(1)

tick():
    if alive_state == false:
        return
```

```

    if fault_mode == NONE:
        update_normal_state()
    else:
        apply_fault_state()

    if fault_end_time exists and now >= fault_end_time:
        recover()

get_status():
    return current_state

inject_fault(type, duration):
    fault_mode = type
    fault_end_time = now + duration
    apply_fault_state()

recover():
    fault_mode = NONE
    restore_normal_state()

reset():
    initialize_state()

```

1-12. 이 프로세스가 하지 않는 것

헛갈리면 안 되는 부분이다. Host Simulator는 아래를 하지 않는다.

- 이벤트 메시지 생성 안 함
- 네트워크 전송 안 함
- ACK 처리 안 함
- retry 안 함
- dedup 안 함
- failover 안 함

그건 전부 다음 프로세스들 역할이다.

Host Simulator는 오직:

- 상태 생성
- 장애 생성
- 복구 생성
- 상태 제공

만 한다.

1-13. 이 프로세스가 꼭 있어야 하는 이유

이 프로세스가 없으면 Local Agent가 읽을 대상이 없다.

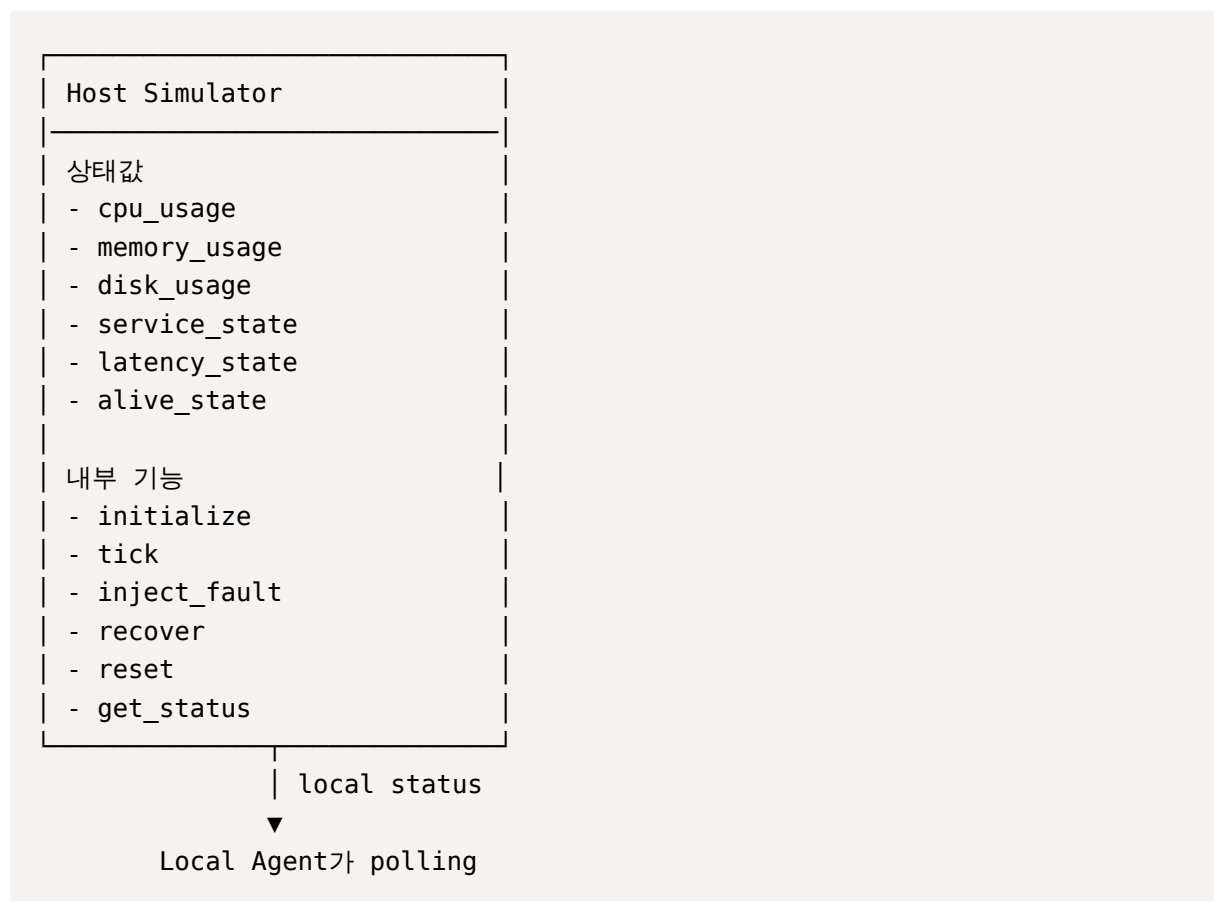
그러면 전체 프로젝트가 단순 네트워크 전송 실험으로만 남는다.

Host Simulator가 있어야:

- “호스트 상태 변화”가 생기고
- Agent가 그것을 이벤트로 만들고
- Relay가 전달하고
- Monitor가 시각화한다

즉 전체 파이프라인의 **출발점**이다.

1-14. Host Simulator만 따로 구조도로 그리면



1-15. 지금 단계에서 이해해야 할 핵심

Host Simulator는 다음 한 문장으로 정리된다.

Host Simulator는 실제 서버 대신 CPU, 메모리, 서비스 상태와 장애/복구 상태를 만들어내고, Local Agent가 읽을 수 있도록 현재 상태를 제공하는 프로세스다.